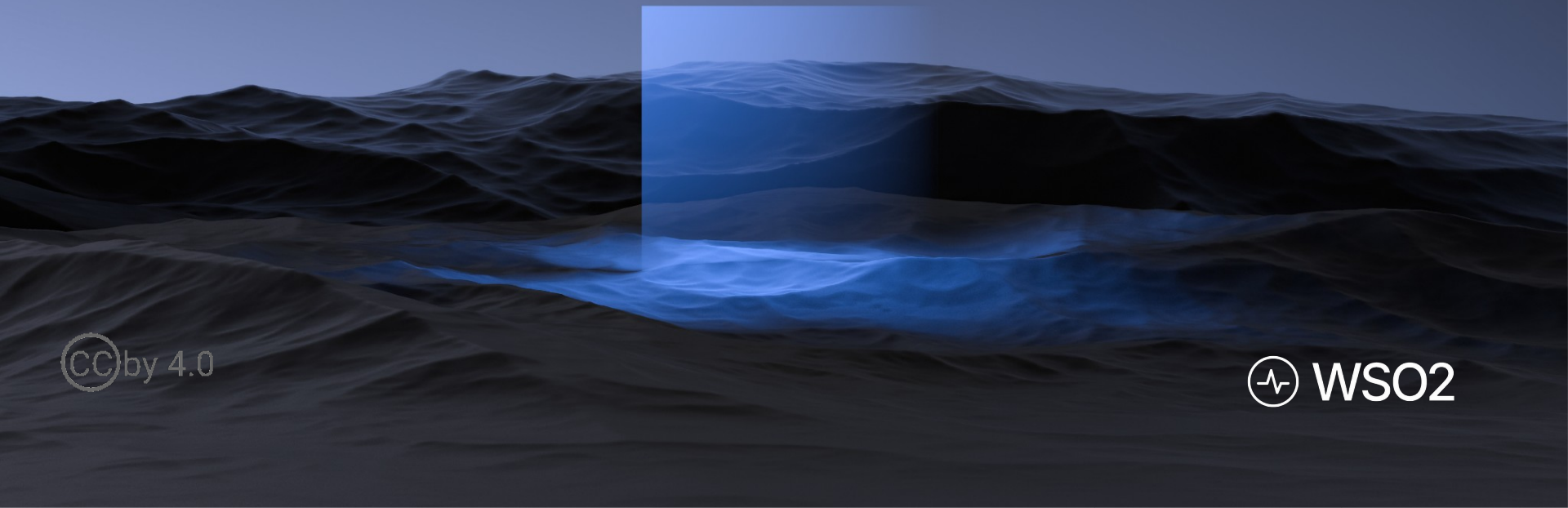
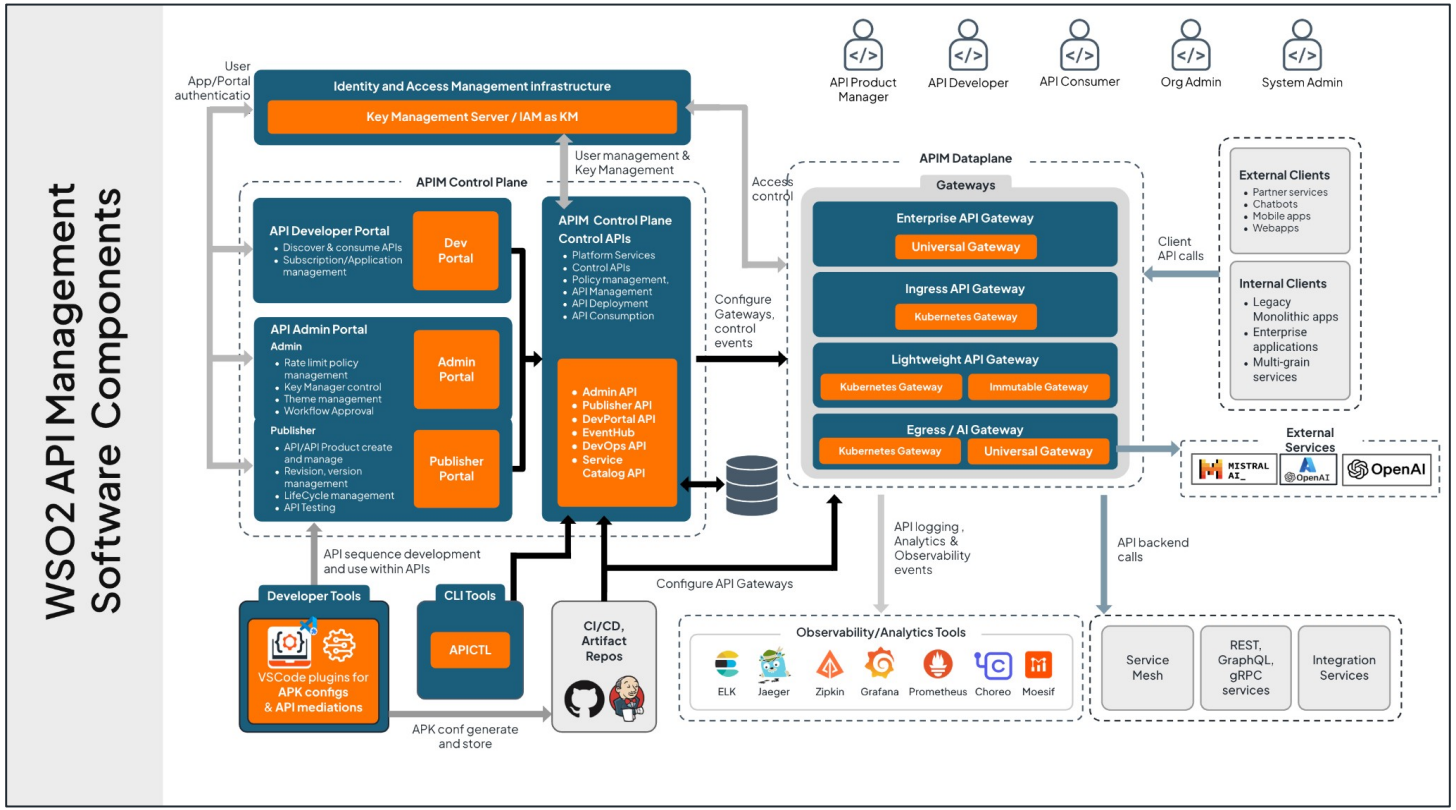
Architecture



CC by 4.0

WSO2

Understanding the API-M Architecture



API Manager Architecture

Control Plane	Control plane is where the API creation and management is done. This consists of portals (publisher, developer portal, admin portal) for users to create and manage APIs, implement rate limiting policies, monitoring and monetization etc. It also provides a set of APIs to interact with external tools like API Controller.
Data Plane	Data plane is where the created API is exposed to the public/ consumers and acts as the proxy for API calls. This also provides additional capabilities such as enforcing security, rate limiting etc. (policy enforcement point).

Understanding the API-M Components



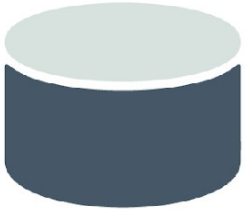
APIM Architecture

API Manager Components and Functionality

Component	Functionality
Publisher	Enables API providers to easily deploy and publish their APIs, share documentation, provision API keys, and gather feedback on API features, quality, and usage.
Developer Portal	Enables consumers to self-register, discover API functionality, subscribe to APIs, evaluate them, and interact with API publishers.
Key Manager	Responsible for all security and key-related operations.
Classic Gateway	Responsible for securing, protecting, managing, and scaling API calls.
Traffic Manager	Responsible for making rate limiting decisions.

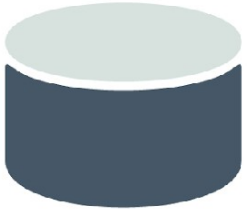
Data Storages

AM database
(WSO2AM_DB)



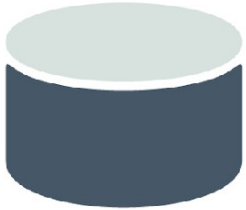
API Manager Specific Data

Shared database
(WSO2SHARED_DB)



Registry and User Management Data

Carbon database
(WSO2CARBON_DB)



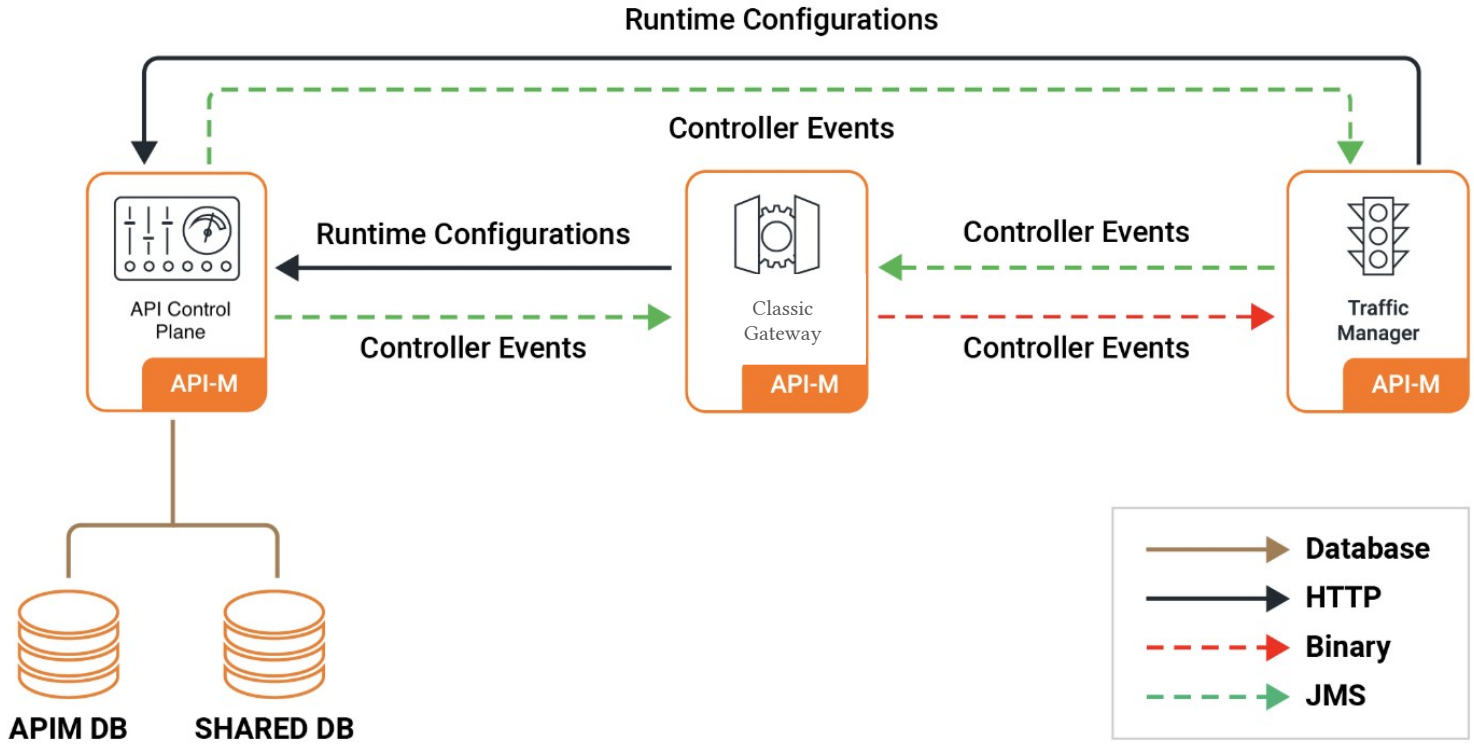
Server Specific Internal Data

Working with Databases

API Manager uses three databases

Datasource	Usage
API Manager Database (WSO2AM_DB)	• Stores information related to the APIs along with the API subscription details. • The Key Manager uses this database to store user access tokens that are used for verification of API calls.
User Manager Database & Registry Database (WSO2SHARED_DB)	• Stores information related to users and user roles. • This information is shared among the Key Manager, Developer Portal and Publisher which resides within the Control Plane. • Users can access the Publisher for API creation and the Developer Portal for consuming the APIs.
	• Shares information between the Publisher and Developer Portal. • When an API is published through the Publisher, it is made available in the Developer Portal via the registry database. • If you are planning to create this setup for a multi-tenanted environment, it is required to share the information in this database between the Gateway and Key Manager components as well.

Recommended Distributed Deployment



Component	Functionality
Classic Gateway	• Responsible for securing, protecting, managing, and scaling API calls. • The universal gateway is a simple API proxy that intercepts API requests and applies policies such as rate limiting and security checks. • It is instrumental in gathering API usage statistics. • A set of handlers are used for security validation and rate limiting purposes in the Gateway. Upon validation, it will pass Web service calls to the actual back end.
Control Plane	• Publisher portal enables API providers to easily publish and deploy their APIs, share documentation, provision API keys, and gather feedback on API features, quality, and usage. • You can create new APIs by pointing to the actual back end service and also define rate-limiting policies available for the API and business policies to various operations. • Developer portal provides a space for consumers to self-register, discover API functionality, subscribe to APIs, evaluate them, and interact with API publishers. • Users can view existing APIs and create their own application by bundling multiple APIs together into one application. • Key Manager component is responsible for all security and key-related operations.

Component	Functionality
RDBMS	The two databases: 1. API Manager Database 2. Shared Database
Traffic Manager	• This is used to make a decision on rate limiting. • Consists of CEP and MB components • Rate limiting decisions are taken by using execution plans • Rate limiting decisions are published to a JMS topic. • Gateway nodes subscribes to this topic to get the already rate limited messages.

Publishing and Deploying APIs

- A user with the Internal/publisher role can publish and deploy APIs via the Publisher server.
- When an API is deployed, the Gateway must be updated with this API so that users can invoke it.
- In a clustered gateway, all Gateway server nodes in the cluster are updated with the published API details, enabling any of these Gateway nodes to serve API calls that are received.
- When an API is published, it is also pushed to the shared database so that it can be made available on the developer portal.

Subscribing to APIs

- A user with the subscriber role logs into the API Developer Portal and subscribes to an API.
- The user must then generate an access token to be able to invoke the API.
- When the subscriber requests to generate the token, a request is sent to the Key Manager component.
- The token is then generated, and the access token details are displayed to the subscriber via the Developer Portal.

Invoking APIs

- Subscribed users can invoke an API to which they have subscribed.
- When the API is invoked, the request is sent to the Gateway server cluster.
- The Gateway server then verifies the token.
- Once the request is verified, the Gateway connects to the back-end implementation and obtains the response, which is sent back to the subscriber via the Gateway server.